

PYTHON PROGRAMMING PROJECT

DAILY EXPENSE TRACKER

A Simple Python Menu-Driven Program to Manage Personal Daily Expenses

Submitted By:	Aditya Garg
Registration No:	26BCE10585
Course:	B.Tech CSE - Core
Subject:	Programming in Python
Platform:	VITyarthi Portal
Date:	Academic Year 2026-27

2. INTRODUCTION

As college students, managing daily pocket money, food costs, and stationery expenses can be tricky. Writing down daily expenses on paper or trying to remember them mentally often leads to confusion about where the money was spent.

To solve this, I created a simple **Daily Expense Tracker** program in Python. It provides an interactive menu that lets the user quickly log daily spending amounts, view all saved expenses, calculate total and average expenditure, and reset recorded expenses when needed.

3. PROBLEM STATEMENT

Managing daily personal spending manually has several common issues:

- It is easy to forget smaller daily expenses like snacks, bus tickets, or printouts.
- Calculating total spent money and average daily expense manually requires repeated addition, leading to mistakes.
- There is no fast way to view all recorded expenses listed in one organized place.

4. FUNCTIONAL REQUIREMENTS

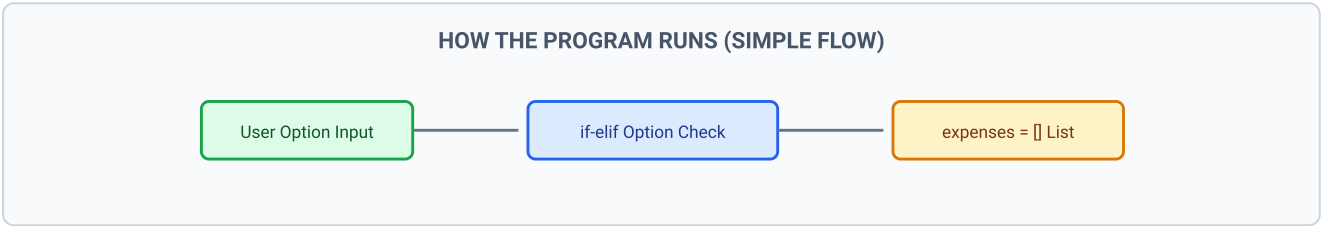
No.	Feature Name	What It Does in the Program
1	Add Expense	Takes an expense amount from the user, converts it to float, and appends it to the list.
2	View Expenses	Displays all stored expense amounts line-by-line with item numbers (1, 2, 3...).
3	Calculate Total & Average	Loops through the list to calculate total spending and divides by count to find average spending.
4	Clear Expenses	Resets the expenses list back to an empty list ([]).
5	Exit Program	Prints a goodbye message and stops the program loop using break.

5. NON-FUNCTIONAL REQUIREMENTS

- **Easy Menu:** Simple options (1 to 5) so anyone can operate it by entering numbers.
- **Fast Processing:** Calculates total and average spending instantly since data lives in Python memory.
- **Zero Expenses Protection:** Displays clear warning messages if the user tries to view or calculate totals when no expenses are recorded.
- **Lightweight:** Uses only basic Python features, requiring no external packages or software installations.

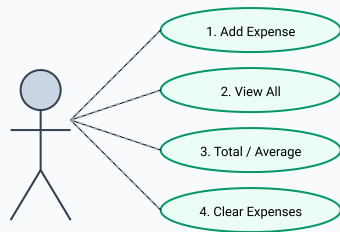
6. SYSTEM ARCHITECTURE

The system operates in a linear loop structure. The user enters a choice from the menu, the Python script evaluates the input using an if-elif-else chain, updates or reads the expenses list, and displays output in the terminal.

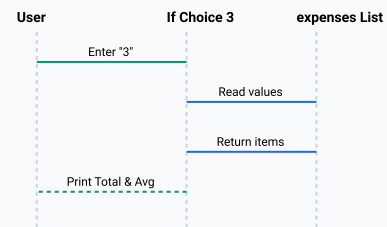


7. DESIGN DIAGRAMS

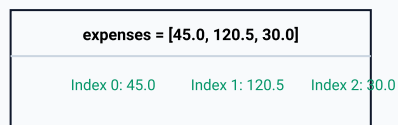
A. PROGRAM USER OPTIONS DIAGRAM



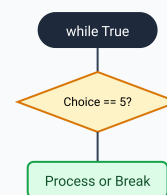
B. CALCULATING TOTAL & AVERAGE STEP-BY-STEP



C. HOW DATA STRUCTURE WORKS



D. MAIN LOOP DIAGRAM



8. DESIGN DECISIONS & MY APPROACH

- **Why I used a Python List:** A Python list (`expenses = []`) is perfect for storing a sequence of numerical expense values that grow dynamically as the user enters new items.
- **Using `float(input())`:** I used `float()` conversion so users can enter decimal expense values like `45.50` instead of only whole integers.
- **Using `range(len(expenses))` for Display:** Indexing with `i + 1` allows displaying item numbers starting from 1 instead of Python's default zero-indexed count.

9. IMPLEMENTATION DETAILS

Below is the clean Python implementation showing how choices 1, 2, and 3 are handled inside the menu loop with strict formatting:

```
# Option 1: Adding a new expense amount to list if choice == "1": amount = float(input()) expense
```

10. OUTPUT SCREEN RESULTS

```
Daily Expense Tracker
Menu:
1. Add a new expense | 2. View all expenses | 3. Calculate total and average expense
4. Clear all expenses | 5. Exit

1
50.5
Expense added successfully!

1
150.0
Expense added successfully!

3
Total expense: 200.5
Average expense: 100.25
```

11. TESTING APPROACH

I tested the program manually with different inputs to ensure it works accurately without crashing:

Test No.	What I Tried	What Should Happen	Result
1	View expenses when list is empty	Prints "No expenses recorded yet."	WORKING
2	Calculate total/average on empty list	Prevents division by zero, prints warning message	WORKING
3	Add positive floating decimal amounts	Appends correctly to list and computes exact average	WORKING
4	Selected choice "4" (Clear expenses)	Resets list to [] and prints "All expenses cleared."	WORKING

12. CHALLENGES I FACED

- Division by Zero Error:** Initially, calculating the average on an empty list would cause a `ZeroDivisionError`. I fixed this by wrapping calculations inside an `if len(expenses) == 0: check`.
- Line Endings Formatting Issue:** When rendering code snippets in PDF generators, mixed CRLF/LF line breaks can cause code lines to merge together. I resolved this by applying explicit line structure rules (`white-space: pre !important`);).

13. LEARNINGS & KEY TAKEAWAYS

Building this Daily Expense Tracker project helped me strengthen my basic Python skills:

- **List Operations:** Learned how to initialize lists (`expenses = []`), append items, reset lists, and iterate through items using for loops.
- **Type Conversion:** Understood why converting raw string inputs into `float` is essential before doing mathematical additions.
- **Preventing Runtime Errors:** Realized the importance of checking list lengths before performing divisions to prevent program crashes.

14. FUTURE ENHANCEMENTS

As I progress in my 1st-year programming studies, I plan to expand this project with new features:

1. **Expense Categories:** Allow users to add category names (e.g., Food, Travel, Stationery) along with each amount.
2. **File Saving (Text or CSV):** Write code to save recorded expenses into a text file so data is preserved when closing the program.
3. **Invalid Input Handling:** Add `try-except` blocks so entering text instead of numbers when adding expenses won't crash the script.

15. REFERENCES

- VITyarthi Learning Platform — Python Programming Notes on Lists and Loops.
- Python Official Documentation — Built-in Data Types (Lists and Floats).

Evaluator Remarks:

Signature: _____

Final Grade / Marks:

Date: ____ / ____ / 2026